

Laporan Tugas Kecil 2 IF2211 Strategi Algoritma
Semester II Tahun 2025/2026
Voxelization Objek 3D menggunakan Octree



Anggota Kelompok:

Daniel Anindito Nugroho / 13524002
Al Farabi / 13524086

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025**

I. Pendahuluan

Voxelization merupakan proses konversi representasi objek tiga dimensi yang bersifat kontinu (umumnya berupa mesh yang tersusun dari titik dan segitiga dalam format .obj) menjadi representasi volumetrik yang diskrit yang disebut voxel. Istilah voxel sendiri merupakan singkatan dari volume element, yang secara visual menyerupai kubus-kubus kecil, serupa dengan estetika visual pada permainan Minecraft.

Dalam tugas kecil ini, proses konversi dilakukan dengan memanfaatkan struktur data Octree. Octree adalah struktur data pohon di mana setiap node internal memiliki tepat delapan anak. Penggunaan Octree sangat efisien dalam pengolahan ruang tiga dimensi karena memungkinkan program untuk membagi ruang secara hierarkis. Dengan menerapkan prinsip spatial partitioning, program hanya akan menelusuri bagian ruang yang bersentuhan dengan permukaan objek dan mengabaikan ruang kosong (empty space). Hal ini secara drastis mengurangi beban komputasi dibandingkan dengan metode grid statis.

Tujuan utama dari program ini adalah mengimplementasikan algoritma Divide and Conquer untuk melakukan surface voxelization. Hasil akhir yang diharapkan adalah sebuah file .obj baru yang tersusun dari kumpulan kubus (voxel) dengan ukuran seragam (uniform) yang diambil dari leaf node pada tingkat kedalaman (depth) maksimum yang ditentukan oleh pengguna.

II. Algoritma

1. Pseudocode

```
Algoritma Voxelization_Octree_Divide_and_Conquer
{ Algoritma untuk mengonversi model 3D menjadi voxel
menggunakan struktur data Octree dengan pendekatan
rekursif Divide and Conquer }

KAMUS
  Type Vector3 : < X, Y, Z : real >
  Type Triangle : < V1, V2, V3 : Vector3 >
  Type BoundingBox : < Min, Max : Vector3 >
  Type Voxel : < Center : Vector3, HalfSize : real >
  Variabel Global:
    MaxDepth : integer
    Voxels : list of Voxel
    NodesPerLevel : array [0..MaxDepth] of integer
    SkippedPerLevel : array [0..MaxDepth] of integer

ALGORITMA
  { Inisialisasi awal di main program }
  Input(pathFile, MaxDepth)
  allTriangles ← ReadOBJ(pathFile)
  rootBox ← CalculateInitialBounds(allVertices)
  { Memanggil prosedur utama }
  CALL Subdivide(rootBox, 0, allTriangles)
```

```
PROCEDURE Subdivide(box : BoundingBox, depth : integer,  
triangles : list of Triangle)
```

KAMUS

```
    intersectingTriangles : list of Triangle  
    tri : Triangle  
    children : array [0..7] of BoundingBox  
    childBox : BoundingBox  
    i : integer
```

ALGORITMA

```
    { 1. CONQUER: Mencari segitiga yang berpotongan dengan  
kotak saat ini }  
    intersectingTriangles  $\leftarrow$   $\emptyset$   
    FOR EACH tri IN triangles DO  
        IF (IsIntersect(tri, box)) THEN  
            intersectingTriangles  $\leftarrow$  intersectingTriangles  $\cup$   
{tri}  
        ENDIF  
    ENDFOR  
    { 2. PRUNING: Jika tidak ada perpotongan (udara  
kosong) }  
    IF (intersectingTriangles =  $\emptyset$ ) THEN  
        SkippedPerLevel[depth]  $\leftarrow$  SkippedPerLevel[depth] + 1  
        EXIT PROCEDURE  
    ENDIF  
  
    { Mencatat statistik node yang berisi permukaan }  
    NodesPerLevel[depth]  $\leftarrow$  NodesPerLevel[depth] + 1  
  
    { 3. BASE CASE: Jika mencapai kedalaman maksimum }  
    IF (depth = MaxDepth) THEN  
        Voxels  $\leftarrow$  Voxels  $\cup$  {NewVoxel(box.GetCenter(),  
box.GetHalfSize())}  
        EXIT PROCEDURE  
    ENDIF  
  
    { 4. DIVIDE: Bagi kotak menjadi 8 anak kotak (oktan) }  
    children  $\leftarrow$  GenerateChildren(box)  
  
    FOR i  $\leftarrow$  0 TO 7 DO  
        childBox  $\leftarrow$  children[i]  
  
        { Rekursi ke level selanjutnya }  
        CALL Subdivide(childBox, depth + 1,  
intersectingTriangles)  
    ENDFOR  
ENDPROCEDURE
```

```
FUNCTION GenerateChildren(box : BoundingBox)  $\rightarrow$  array  
[0..7] of BoundingBox
```

```

KAMUS
    c, min, max : Vector3

ALGORITMA
    c ← box.GetCenter()
    min ← box.Min
    max ← box.Max
    { Menghasilkan 8 kombinasi kotak baru berdasarkan
titik tengah }
    RETURN [
        BoundingBox(min, c), { Kiri-Bawah-Depan }
        BoundingBox(<c.X, min.Y, min.Z>, <max.X, c.Y,
c.Z>), { Kanan-Bawah-Depan }
        BoundingBox(<min.X, c.Y, min.Z>, <c.X, max.Y,
c.Z>), { Kiri-Atas-Depan }
        BoundingBox(<c.X, c.Y, min.Z>, <max.X, max.Y,
c.Z>), { Kanan-Atas-Depan }
        BoundingBox(<min.X, min.Y, c.Z>, <c.X, c.Y,
max.Z>), { Kiri-Bawah-Belakang }
        BoundingBox(<c.X, min.Y, c.Z>, <max.X, c.Y,
max.Z>), { Kanan-Bawah-Belakang }
        BoundingBox(<min.X, c.Y, c.Z>, <c.X, max.Y,
max.Z>), { Kiri-Atas-Belakang }
        BoundingBox(c, max) { Kanan-Atas-Belakang }
    ]
ENDFUNCTION

```

2. Implementasi Step by Step

1) Filter (Conquer)

Setiap kali fungsi subdivide dipanggil, langkah pertama adalah menyaring segitiga-segitiga yang ada. Program tidak mengecek seluruh segitiga model asli, melainkan hanya segitiga yang diwariskan dari induknya. Fungsi IsIntersect (menggunakan algoritma Akenine-Möller) menentukan apakah permukaan objek melewati volume kotak saat ini.

2) Pruning

Jika hasil penyaringan segitiga kosong, berarti kotak tersebut berada di ruang hampa. Program akan berhenti melakukan rekursi pada cabang ini. Langkah ini sangat krusial dalam Divide and Conquer untuk meminimalkan penelusuran pada area yang tidak relevan.

3) Base Case

Ketika rekursi mencapai MaxDepth dan kotak tersebut terdeteksi berisi permukaan objek, kotak tersebut resmi menjadi voxel. Data pusat kotak dan ukurannya disimpan ke dalam daftar hasil akhir. Karena diambil hanya pada MaxDepth, ukuran seluruh voxel akan seragam (uniform).

4) Divide

Jika kotak masih berisi segitiga namun belum mencapai kedalaman maksimum, kotak induk dibagi menjadi 8 anak kotak melalui fungsi

generateChildren. Setiap anak kotak mewakili satu oktan (Kiri-Bawah-Depan, Kanan-Bawah-Depan, dst).

5) Optimasi Konkurensi

Untuk mempercepat pemrosesan, program menggunakan multithreading pada level-level atas (ketika $\text{depth} < 3$). Dengan memproses sub-ruang secara paralel, beban komputasi terbagi ke berbagai core CPU. Penggunaan `sync.Mutex` (`v.mu.Lock()`) memastikan penulisan data statistik dan hasil tetap aman dari race condition.

3. Source Code

```
func (v *Voxelizer) Build(rootBox BoundingBox) {
    var wg sync.WaitGroup
    v.subdivide(rootBox, 0, v.Triangles, &wg, true)
    wg.Wait()
}

func (v *Voxelizer) subdivide(box BoundingBox, depth int,
    triangles []Triangle, wg *sync.WaitGroup, isParallel bool) {
    intersectingTriangles := []Triangle{}
    for _, tri := range triangles {
        if IsIntersect(tri, box) {
            intersectingTriangles =
                append(intersectingTriangles, tri)
        }
    }

    if len(intersectingTriangles) == 0 {
        v.mu.Lock()
        v.SkippedPerLevel[depth]++
        v.mu.Unlock()
        if isParallel {
            wg.Done()
        }
        return
    }

    v.mu.Lock()
    v.NodesPerLevel[depth]++
    v.mu.Unlock()

    if depth == v.MaxDepth {
```

```

        v.mu.Lock()
        v.Voxels = append(v.Voxels, Voxel{
            Center:    box.GetCenter(),
            HalfSize:  box.GetHalfSize(),
        })
        v.mu.Unlock()
        if isParallel {
            wg.Done()
        }
        return
    }

    children := v.generateChildren(box)
    useGoroutine := depth < 3

    for _, childBox := range children {
        if useGoroutine {
            wg.Add(1)
            go v.subdivide(childBox, depth+1,
                intersectingTriangles, wg, true)
        } else {
            v.subdivide(childBox, depth+1,
                intersectingTriangles, wg, false)
        }
    }

    if isParallel {
        wg.Done()
    }
}

func (v *Voxelizer) generateChildren(box BoundingBox)
[8]BoundingBox {
    center := box.GetCenter()
    min := box.Min
    max := box.Max

    var children [8]BoundingBox
    // 0: Kiri - Bawah - Depan

```

```

        children[0] = BoundingBox{Min: Vector3{min.X, min.Y,
min.Z}, Max: Vector3{center.X, center.Y, center.Z}}

        // 1: Kanan - Bawah - Depan
        children[1] = BoundingBox{Min: Vector3{center.X, min.Y,
min.Z}, Max: Vector3{max.X, center.Y, center.Z}}

        // 2: Kiri - Atas - Depan
        children[2] = BoundingBox{Min: Vector3{min.X, center.Y,
min.Z}, Max: Vector3{center.X, max.Y, center.Z}}

        // 3: Kanan - Atas - Depan
        children[3] = BoundingBox{Min: Vector3{center.X, center.Y,
min.Z}, Max: Vector3{max.X, max.Y, center.Z}}

        // 4: Kiri - Bawah - Belakang
        children[4] = BoundingBox{Min: Vector3{min.X, min.Y,
center.Z}, Max: Vector3{center.X, center.Y, max.Z}}

        // 5: Kanan - Bawah - Belakang
        children[5] = BoundingBox{Min: Vector3{center.X, min.Y,
center.Z}, Max: Vector3{max.X, center.Y, max.Z}}

        // 6: Kiri - Atas - Belakang
        children[6] = BoundingBox{Min: Vector3{min.X, center.Y,
center.Z}, Max: Vector3{center.X, max.Y, max.Z}}

        // 7: Kanan - Atas - Belakang
        children[7] = BoundingBox{Min: Vector3{center.X, center.Y,
center.Z}, Max: Vector3{max.X, max.Y, max.Z}}

        return children
    }

```

III. Hasil Pengujian

a. Test case 1 (line)

i. Input:

```
PS C:\Users\alfar\VSCode\sekolah\SMT4\STIMA\tucil\tucil2\tucil2_13524002_13524086\src> go run .
Masukkan path file .obj (contoh: ../test/cow.obj): ../test/line.obj
Berhasil load 1 segitiga dan 3 vertex
Bounding Box=== Min: (2.21, -0.99, -0.97), Max: (2.43, -0.78, -0.75)
Depth octree (default 6):

Memproses octree depth 6...
Menyimpan hasil ke ../test/line_voxelized_d6.obj...

=[]=[]=[]= REPORTS =[]=[]=[]=
File input: ../test/line.obj
File output: ../test/line_voxelized_d6.obj

Total voxel:      1005
Total vertex:    8040 (voxel x 8)
Total face:     12060 (voxel x 12)

Waktu proses: 1.8508ms

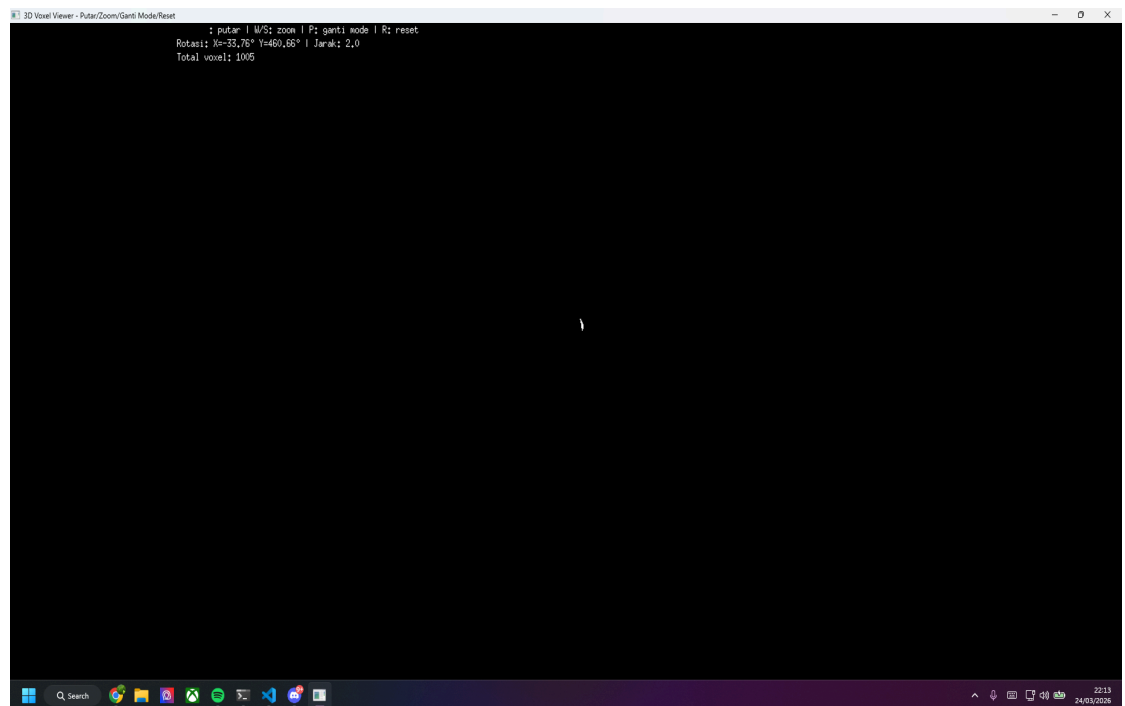
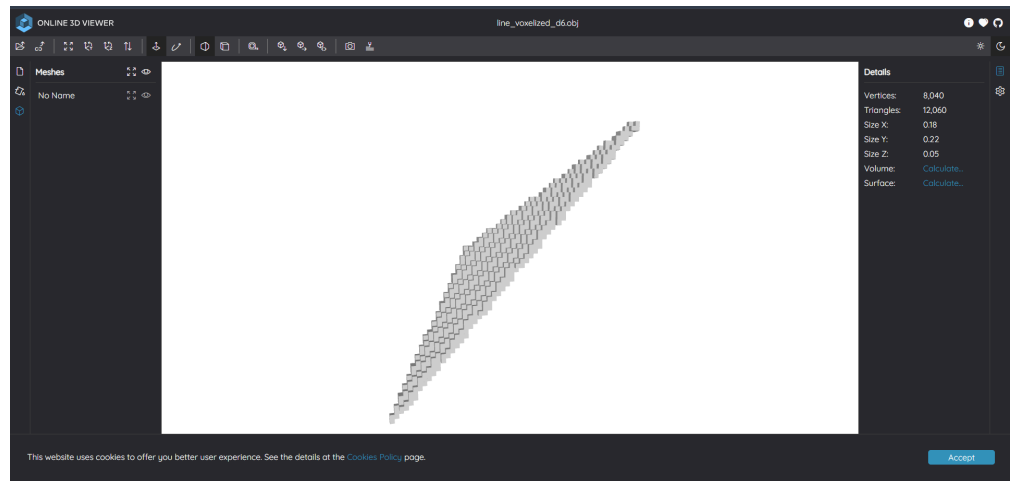
Statistik per level:
Level | Dibuat | Di-skip
-----|-----|-----
0 | 1 | 0
1 | 7 | 1
2 | 15 | 41
3 | 32 | 88
4 | 91 | 165
5 | 284 | 444
6 | 1005 | 1267

Apakah ingin melihat model dalam 3D viewer? (y/n): y
Membuka viewer...
Kontrol: Arrow Keys=putar, W/S=zoom, P=ganti mode, R=reset
```

ii. Output:

test > line_voxelized_d6.obj

```
1 v 2.245307 -0.959910 -0.871918
2 v 2.248695 -0.959910 -0.871918
3 v 2.248695 -0.956522 -0.871918
4 v 2.245307 -0.956522 -0.871918
5 v 2.245307 -0.959910 -0.868530
6 v 2.248695 -0.959910 -0.868530
7 v 2.248695 -0.956522 -0.868530
8 v 2.245307 -0.956522 -0.868530
9 f 1 2 3
10 f 1 3 4
11 f 5 6 7
12 f 5 7 8
13 f 1 5 8
14 f 1 8 4
15 f 2 6 7
16 f 2 7 3
17 f 4 3 7
18 f 4 7 8
19 f 1 2 6
20 f 1 6 5
21 v 2.245307 -0.956522 -0.871918
22 v 2.248695 -0.956522 -0.871918
23 v 2.248695 -0.953133 -0.871918
24 v 2.245307 -0.953133 -0.871918
25 v 2.245307 -0.956522 -0.868530
26 v 2.248695 -0.956522 -0.868530
27 v 2.248695 -0.953133 -0.868530
28 v 2.245307 -0.953133 -0.868530
29 f 9 10 11
30 f 9 11 12
31 f 13 14 15
32 f 13 15 16
33 f 9 13 16
34 f 9 16 12
35 f 10 14 15
36 f 10 15 11
37 f 12 11 15
38 f 12 15 16
39 f 9 10 14
40 f 9 14 13
41 v 2.248695 -0.956522 -0.871918
42 v 2.252084 -0.956522 -0.871918
43 v 2.252084 -0.953133 -0.871918
44 v 2.248695 -0.953133 -0.871918
```



b. Test case 2 (cow)

i. Input:

```

PS C:\Users\alfar\VSCode\sekolah\SMT4\STIMA\tucil\tucil2\Tucil2 13524002 13524006\src> go run .
Masukkan path file .obj (contoh: ../test/cow.obj): ../test/cow.obj
Berhasil load 5804 segitiga dan 4583 vertex
Bounding Box== Min: (-4.50, -5.71, -5.27), Max: (6.05, 4.84, 5.27)
Depth octree (default 6):

Memproses octree depth 6...
Menyimpan hasil ke ../test/cow_voxelized_d6.obj...

==[==[==[ REPORTS ==[==[==[
File input: ../test/cow.obj
File output: ../test/cow_voxelized_d6.obj

Total voxel:      5502
Total vertex:    44016 (voxel x 8)
Total face:     66024 (voxel x 12)

Waktu proses: 12.7876ms

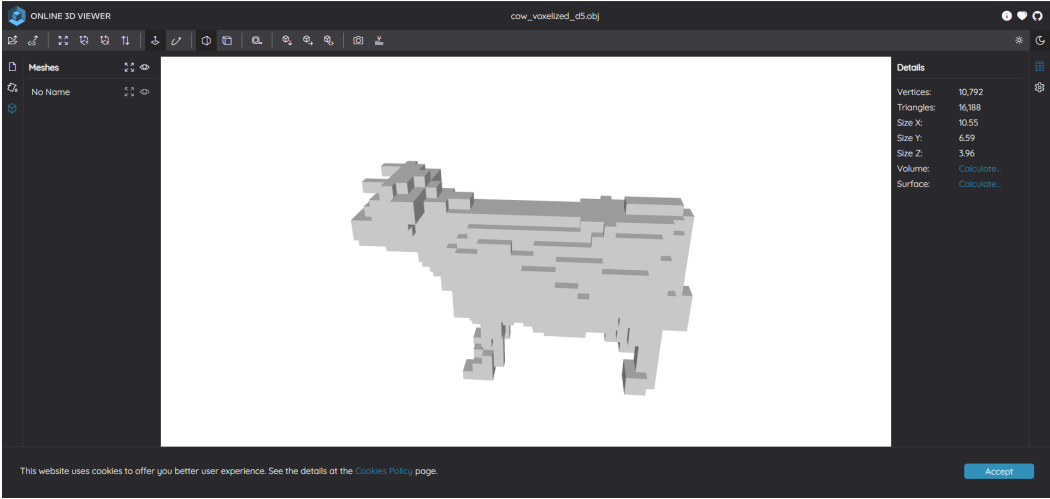
Statistik per level:
Level | Dibuat | Di-skip
-----|-----|-----
0 | 1 | 0
1 | 8 | 0
2 | 20 | 44
3 | 86 | 74
4 | 339 | 349
5 | 1349 | 1363
6 | 5502 | 5290

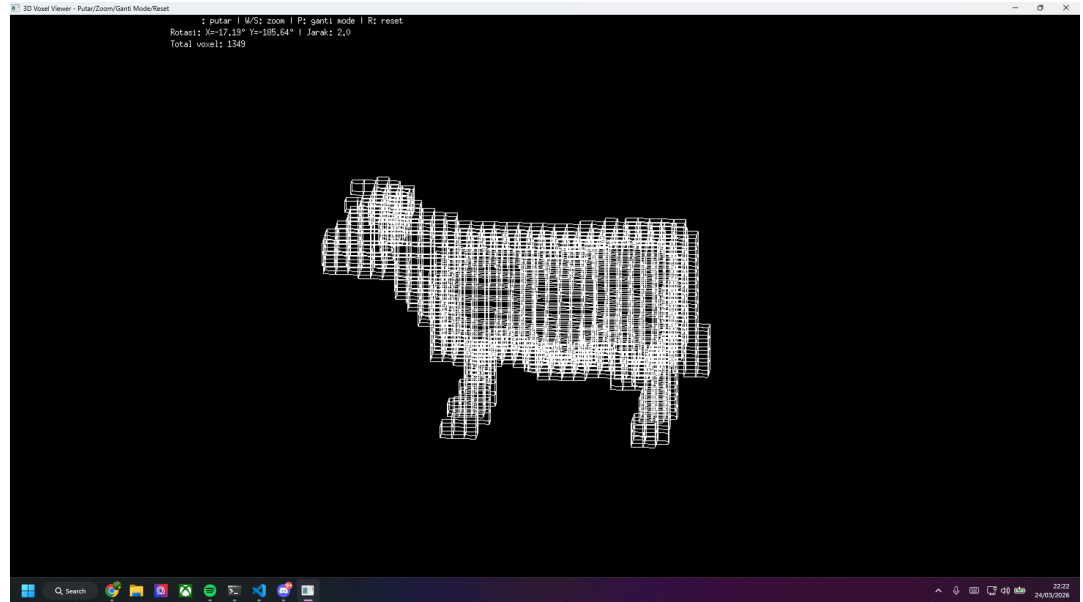
Apakah ingin melihat model dalam 3D viewer? (y/n): y
Membuka viewer...
Kontrol: Arrow Keys=putar, W/S=zoom, P=ganti mode, R=reset

```

ii. Output:

```
test > cow_voxelized_d5.obj
1 v -0.542419 -1.757203 -1.318545
2 v -0.212782 -1.757203 -1.318545
3 v -0.212782 -1.427567 -1.318545
4 v -0.542419 -1.427567 -1.318545
5 v -0.542419 -1.757203 -0.988909
6 v -0.212782 -1.757203 -0.988909
7 v -0.212782 -1.427567 -0.988909
8 v -0.542419 -1.427567 -0.988909
9 f 1 2 3
10 f 1 3 4
11 f 5 6 7
12 f 5 7 8
13 f 1 5 8
14 f 1 8 4
15 f 2 6 7
16 f 2 7 3
17 f 4 3 7
```





c. Test case 3 (pumpkin)

i. Input:

```
PS C:\Users\alfar\VSCode\sekolah\SMT4\STIMA\tucil\tucil2\tucil2_13524002_13524086\src> go run .
Masukkan path file .obj (contoh: ../test/cow.obj): ../test/pumpkin.obj
Berhasil load 10000 segitiga dan 5002 vertex
Bounding Box=== Min: (-42.59, -39.10, -149.98), Max: (37.34, 40.84, -70.05)
Depth octree (default 6): 4

Memproses octree depth 4...
Menyimpan hasil ke ../test/pumpkin_voxelized_d4.obj...

=[]=[]=[]= REPORTS =[]=[]=[]=
File input: ../test/pumpkin.obj
File output: ../test/pumpkin_voxelized_d4.obj

Total voxel: 1102
Total vertex: 8816 (voxel x 8)
Total face: 13224 (voxel x 12)

Waktu proses: 9.9056ms

Statistik per level:
Level | Dibuat | Di-skip
-----|-----|-----
0 | 1 | 0
1 | 8 | 0
2 | 56 | 8
3 | 277 | 171
4 | 1102 | 1114

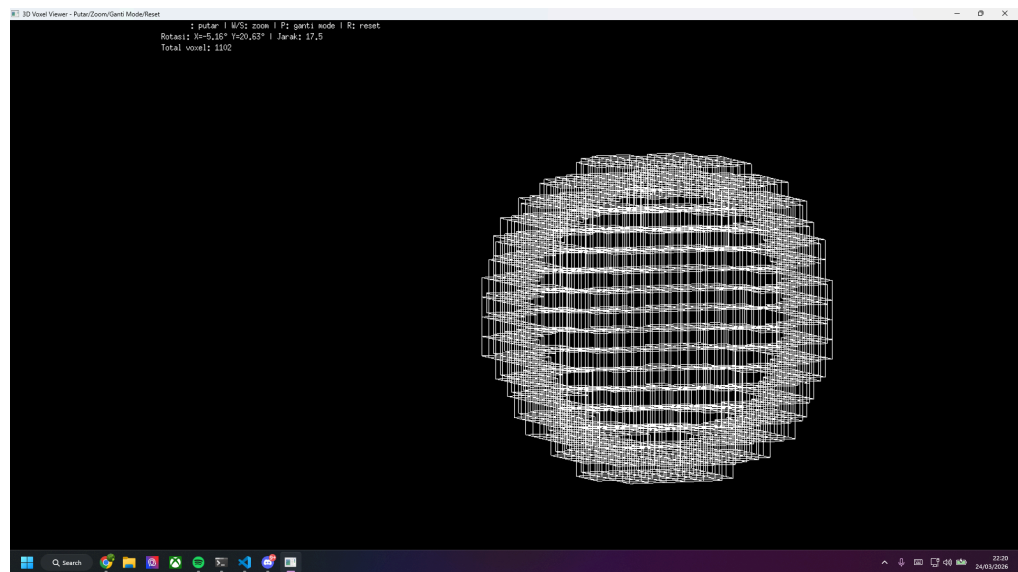
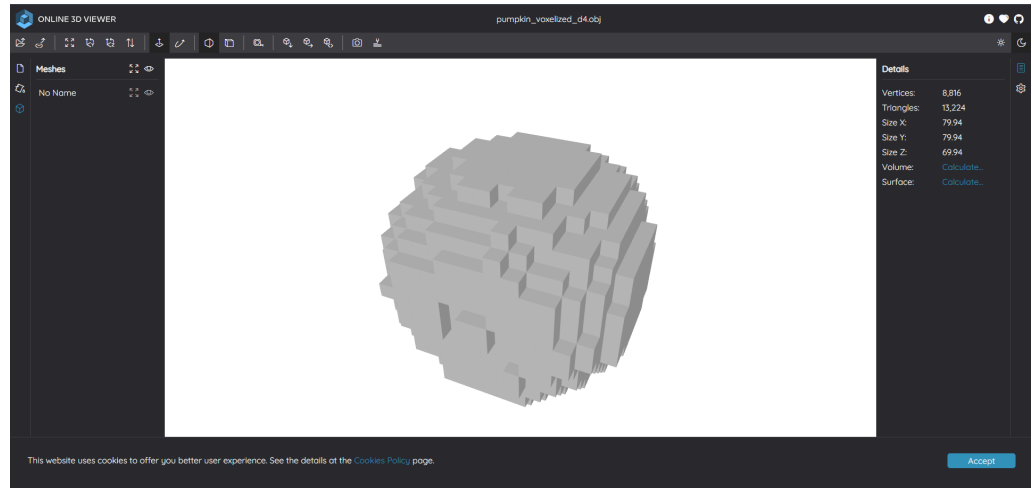
Apakah ingin melihat model dalam 3D viewer? (y/n): y
Membuka viewer...
```

ii. Output:

test >  pumpkin_voxelized_d4.obj

```
1 v -27.603633 -24.109732 -139.991727
2 v -22.607636 -24.109732 -139.991727
3 v -22.607636 -19.113736 -139.991727
4 v -27.603633 -19.113736 -139.991727
5 v -27.603633 -24.109732 -134.995730
6 v -22.607636 -24.109732 -134.995730
7 v -22.607636 -19.113736 -134.995730
8 v -27.603633 -19.113736 -134.995730
9 f 1 2 3
10 f 1 3 4
11 f 5 6 7
12 f 5 7 8
13 f 1 5 8
14 f 1 8 4
15 f 2 6 7
16 f 2 7 3
17 f 4 3 7
```

PROBLEMS **2** OUTPUT DEBUG CONSOLE TERMINAL PORTS



- d. Test case 4 (teapot)
 - i. Input:

```

PS C:\Users\alfar\VSCode\sekolah\SMT4\STIMA\tucil\tucil2\Tucil2_13524002_13524086\src> go run .
Masukkan path file .obj (contoh: ../test/cow.obj): ../test/teapot.obj
Berhasil load 1024 segitiga dan 530 vertex
Bounding Box=== Min: (-85.36, -87.38, -93.38), Max: (99.19, 97.17, 91.17)
Depth octree (default 6): 4

Memproses octree depth 4...
Menyimpan hasil ke ../test/teapot_voxelized_d4.obj...

=[]=[]= REPORTS =[]=[]=
File input: ../test/teapot.obj
File output: ../test/teapot_voxelized_d4.obj

Total voxel:      414
Total vertex:    3312 (voxel x 8)
Total face:     4968 (voxel x 12)

Waktu proses: 2.7283ms

Statistik per level:
Level | Dibuat | Di-skip
-----|-----|-----
0 | 1 | 0
1 | 8 | 0
2 | 24 | 40
3 | 104 | 88
4 | 414 | 418

Apakah ingin melihat model dalam 3D viewer? (y/n): y
Membuka viewer...
Kontrol: Arrow Keys=putar, W/S=zoom, P=ganti mode, R=reset

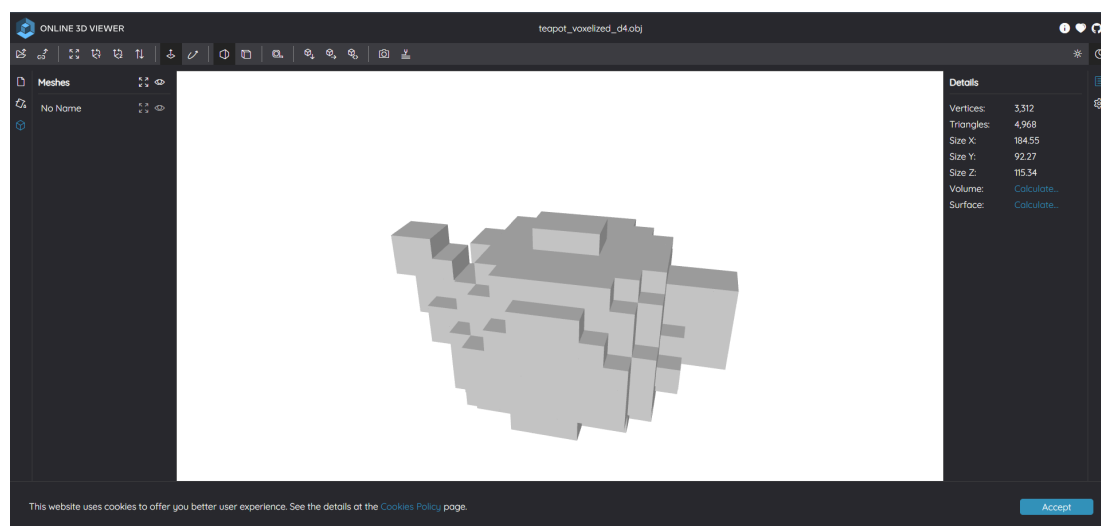
```

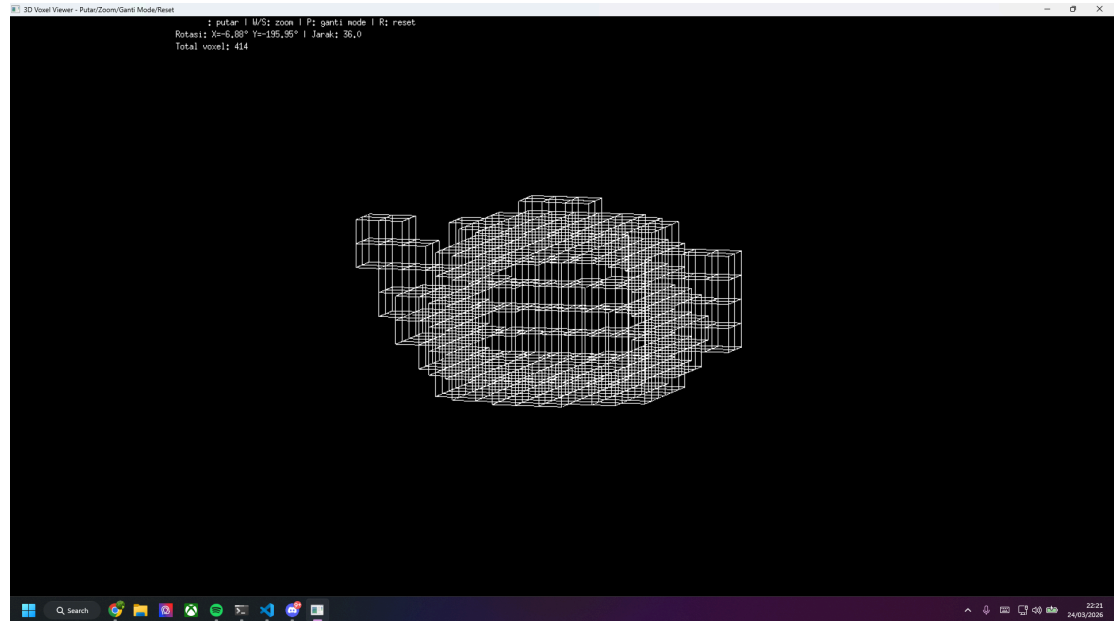
ii. Output:


```

test > teapot_voxelized_d4.obj
1 v 53.053155 -29.708367 -24.176728
2 v 64.587494 -29.708367 -24.176728
3 v 64.587494 -18.174028 -24.176728
4 v 53.053155 -18.174028 -24.176728
5 v 53.053155 -29.708367 -12.642389
6 v 64.587494 -29.708367 -12.642389
7 v 64.587494 -18.174028 -12.642389
8 v 53.053155 -18.174028 -12.642389
9 f 1 2 3
10 f 1 3 4
11 f 5 6 7
12 f 5 7 8
13 f 1 5 8
14 f 1 8 4
15 f 2 6 7
16 f 2 7 3
17 f 4 3 7
18 f 4 7 8
19 f 1 2 6
20 f 1 6 5
21 v 53.053155 -29.708367 -12.642389
22 v 64.587494 -29.708367 -12.642389
23 v 64.587494 -18.174028 -12.642389
24 v 53.053155 -18.174028 -12.642389
25 v 53.053155 -29.708367 -1.108050

```





- e. Test case 5 (input salah)
 - i. Input & output:

```
test > test_invalid_opcode.obj
1 v 0.0 0.0 0.0
2 v 1.0 1.0 1.0
3 v 2.0 2.0 2.0
4 v 3.0 3.0 3.0
5 a 2.520417 -0.954785 -0.739445
6 f 1 2 3
7 |
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Berhasil load 5804 segitiga dan 4583 vertex

PS C:\Users\alfar\VSCode\sekolah\SMK4\STIMA\tucil\tucil2\tucil2_13524002_13524086\src> go run .
 Masukkan path file .obj (contoh: ../test/cow.obj): ../test/cow.obj
 Error baca file: PARSE ERROR Baris 164:
 index out of range (vertex tidak ada)
 max vertex: 65, tapi dapat: 55 66 56

PS C:\Users\alfar\VSCode\sekolah\SMK4\STIMA\tucil\tucil2\tucil2_13524002_13524086\src> go run .
 Masukkan path file .obj (contoh: ../test/cow.obj): ../test/test_invalid_opcode.obj
 Error baca file: PARSE ERROR Baris 8:
 opcode tidak valid: 'a'
 hanya bisa 'v' (vertex) atau 'f' (face)
 data: [a 2.520417 -0.954785 -0.739445]

PS C:\Users\alfar\VSCode\sekolah\SMK4\STIMA\tucil\tucil2\tucil2_13524002_13524086\src>

IV. Kesimpulan

Implementasi strategi algoritma Divide and Conquer pada struktur data Octree terbukti sangat efektif untuk melakukan proses voxelization pada permukaan objek tiga dimensi. Dengan

membagi ruang secara hierarkis dan menerapkan teknik pruning secara rekursif, program mampu melokalisasi permukaan model dengan efisien tanpa harus membuang sumber daya pada bagian ruang yang kosong. Hal ini menunjukkan bahwa partisi ruang yang cerdas melalui prinsip spatial partitioning dapat menekan beban komputasi secara signifikan dengan hanya mewariskan daftar segitiga yang relevan dari induk ke setiap anak kotaknya.

Keberhasilan sistem dalam menghasilkan voxel berukuran seragam (uniform) tercapai dengan membatasi penyimpanan data hanya pada leaf node di tingkat kedalaman maksimum yang telah ditentukan oleh pengguna. Tingkat kedalaman ini menjadi parameter penentu antara tingkat kehalusan detail model dengan beban kerja CPU, di mana kedalaman yang lebih tinggi menghasilkan representasi volumetrik yang lebih mendekati bentuk asli model namun tetap mempertahankan struktur grid yang konsisten. Melalui pendekatan ini, representasi objek yang tadinya bersifat kontinu dalam format mesh berhasil diubah menjadi representasi diskrit yang rapi dan terstruktur.

Akurasi dari surface voxelization ini didukung oleh penggunaan algoritma Triangle-Box Intersection yang presisi dalam mendeteksi perpotongan antara permukaan miring objek dengan kotak-kotak oktan. Selain itu, optimalisasi melalui fitur konkurensi pada tahap awal subdivisi ruang memungkinkan pemanfaatan multi-core CPU secara maksimal sehingga waktu eksekusi total menjadi jauh lebih cepat. Secara keseluruhan, program ini telah berhasil memenuhi seluruh spesifikasi teknis dan bonus yang diminta, serta memberikan bukti melalui laporan statistik CLI bahwa strategi penelusuran ruang yang dilakukan telah berjalan secara optimal.

V. Pranala Repository

https://github.com/le1n-gif/Tucil2_13524002_13524086.git

VI. Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan <i>concurrency</i>	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	

VII. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, consisting of a stylized 'A' followed by a horizontal line.

Al Farabi

A handwritten signature in black ink, featuring a large 'D' followed by 'an' and a stylized 'N'.

Daniel Anindito Nugroho